

PROGRAM

```
class Display {  
    public synchronized void print(String msg) {  
        System.out.print "["+msg);  
        try {  
            Thread.sleep(1000);  
        } catch (Exception e) {  
            System.out.println(e.getMessage());  
        }  
        System.out.println("]");  
    }  
}  
  
class SyncThread extends Thread {  
    private Display d;  
    private String msg;  
    public SyncThread(Display d, String msg) {  
        this.d=d;  
        this.msg = msg;  
    }  
    public void run() {  
        d.print(msg);  
    }  
}  
  
class Test {  
    public static void main(String args[]) {  
        Display d = new Display();  
        SyncThread t1 = new SyncThread(d,"Hello");
```

```
        SyncThread t2 = new SyncThread(d,"World");  
        t1.start();  
        t2.start();  
    }  
}
```

OUTPUT

[Done] exited with code=1 in 0.592 seconds

[Running] cd "c:\Users\HP\Desktop\LAB\JDBC\src\" && javac Test.java && java Test
[Hello]
[World]

[Done] exited with code=0 in 2.713 seconds

ALGORITHM

Synchronized Printing in Multi-threading

1. Initialize Display Class:

- Define a class Display that contains a synchronized method print(String msg).
- Inside the print method:
 - Print the message inside brackets [.
 - Pause execution for 1000 milliseconds (1 second) to simulate a delay.
 - Print the closing bracket].

2. Define SyncThread Class:

- Create a class SyncThread that extends Thread.
- Include:
 - A private member of type Display to hold a reference to a Display object.
 - A private String member to hold the message to print.
- Implement the constructor to initialize the Display object and the message.
- Override the run method:
 - Call the print method on the Display object with the message.

3. Main Method in Test Class:

- Define the Test class with the main method.
- Inside the main method:
 - Create an instance of Display.
 - Create two instances of SyncThread, passing the Display instance and different messages ("Hello" and "World") to each.
 - Start both threads.

4. Execution Flow:

- When `t1.start()` is called, the run method of `SyncThread t1` is executed, which calls `d.print("Hello")`.
- Since the print method is synchronized, `t1` acquires the lock on the `Display` instance.
- `t1` prints [Hello, waits for 1 second, and then prints].
- While `t1` is executing, if `t2.start()` is called, `t2` will wait for `t1` to release the lock on the `Display` instance.
- Once `t1` completes execution and releases the lock, `t2` then acquires the lock and executes `d.print("World")`, printing [World and waiting for another second before printing].